

Computer-Use Automation System - Design and Implementation Report

1. Architecture

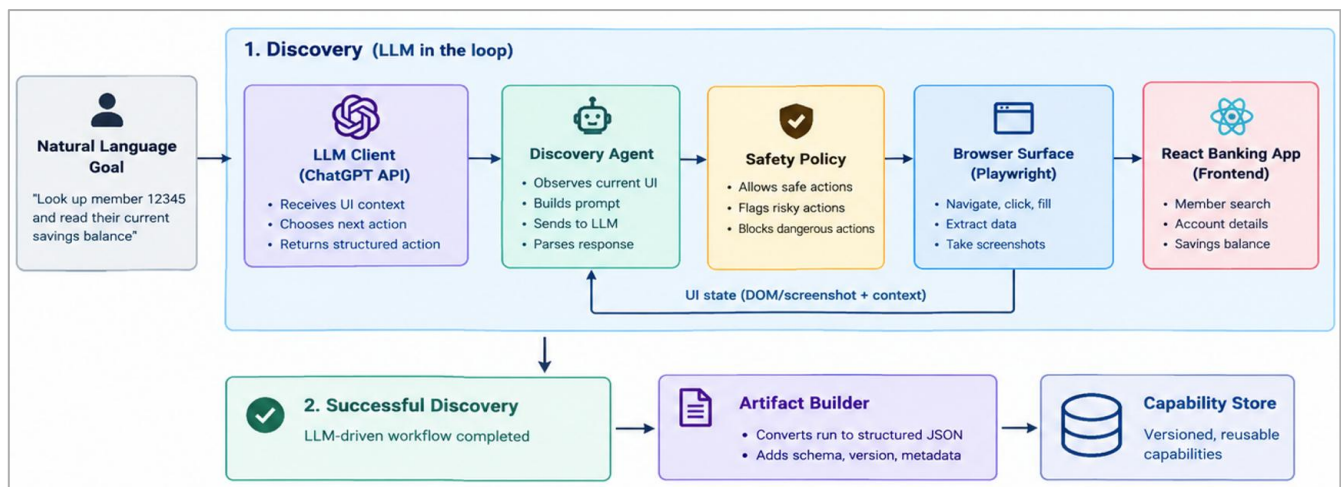
The system allows an AI agent to operate an application through its UI, record successful workflows as reusable capabilities, and replay them deterministically without the LLM..

For the demonstration environment, I built a small **Member Banking Demo** application with two main functionalities: retrieving a member's savings information using their member ID and opening a savings sub-account with an initial balance, which requires human escalation. The frontend is implemented using React, the backend uses FastAPI, and the LLM-driven discovery component uses the ChatGPT/OpenAI API with Playwright for browser automation. The automation system is implemented in Python and is organized into several main components:

1. **Discovery** - The discovery process is model-driven. A **PageObserver** captures the current UI state, including the URL, page text, inputs, buttons, identifiers and labels. This information is provided to the **Discovery Agent**, which builds the LLM prompt and asks the LLM API to determine the next action.

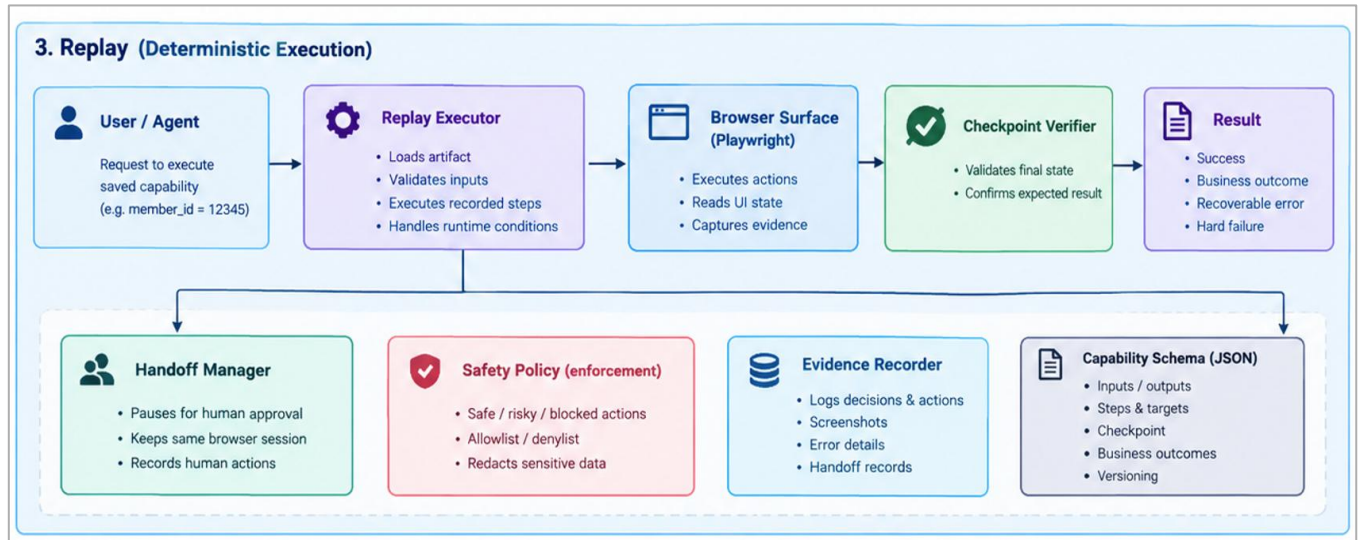
The selected action passes through the **Safety Policy** before being executed through the Playwright browser surface. After execution, the UI is observed again and the process continues until the goal is completed or human intervention is required.

2. **Artifact** - Once the goal is successfully discovered and completed, the **Action Recorder** and **Artifact Builder** convert the successful run into a structured, versioned JSON capability containing the inputs, outputs, recorded steps, targets, checkpoint and relevant metadata



3. **Replay** - Replay is artifact-driven and deterministic. The **ReplayExecutor** loads the saved capability, validates the provided inputs, and executes the recorded steps through the Playwright surface without using the LLM to make decisions.

The **Checkpoint Verifier** confirms that the expected final UI state was reached. Execution results and failures are recorded as logs structured in evidence.



The Architecture separates discovery from execution, enabling deterministic replay without repeatedly using the LLM. This separation was an important design decision because the assignment specifically requires deterministic replay without the LLM remaining in the decision loop.

2. Artifact Schema

A successful discovery is stored as a structured JSON capability. The artifact contains information for both a human reviewer and the replay engine to understand how the capability should be executed. It contains:

- `schema_version` – identify the version
- `capability_id` – identify the reusable capability.
- `description` – describes the business goal.
- `target` – identifies the application and base URL.
- `inputs` – values supplied at replay time.
- `outputs` – values extracted during replay.
- `steps` – ordered actions to execute
- `checkpoint` – observable evidence that proves successful completion.
- `business_outcomes` – legitimate application-level outcomes.
- `metadata` – provenance and discovery information.

The artifact uses a **Target** object to describe how UI elements should be located. Supported strategies include ID, label, role, and text. Targets also include safety metadata indicating whether an action is allowed, requires human approval, or is blocked.

Inputs are parameterized rather than storing discovery-time values directly. For example: `{{member_id}}` This allows the same capability to be replayed with different runtime values.

Two artifacts are generated in this project-

1. **Savings lookup capability** - Discovery is performed using two stages:
 - The first uses a valid member ID to discover and record the **successful execution path**, including the steps required to look up the member and retrieve the savings account balance.
 - The second flow uses a member ID representing a **business outcome**, such as a member that does not exist, and records the legitimate application-level outcome.

The two discovery results are then merged into a **single capability artifact** that supports deterministic replay of both the successful case and the business outcome case.

2. **Open savings sub-account capability** - the discovery process records the steps required to open a savings sub-account for an existing account. During discovery, the approval action is identified as a risky action through the `data-safety="human_required"` attribute. The capability artifact records this safety requirement so that the replay process knows where human intervention is required.

The checkpoint is important because successfully executing a sequence of clicks does not necessarily mean that the business operation is completed. Discovery records an observable final state, and replay verifies that state before returning success.

3. Determinism & Error Handling

The key design decision for deterministic replay is that the LLM is not called during replay. The replay executor loads the saved artifact and executes the predefined steps using Playwright. Determinism is supported by:

1. **Recorded targets** — replay uses the selectors captured during discovery.
2. **Parameterized inputs** — runtime values do not change the workflow itself.
3. **Ordered steps** — the artifact defines the exact execution sequence.
4. **Explicit checkpoints** — checks the expected final UI state is present before success.
5. **Programmatic safety checks** — replay does not blindly trust the artifact.
6. **Evidence recording** — successful and failed executions produce structured evidence.

The replay system separates runtime conditions into three categories.

- **Business outcomes** - A business outcome is an expected application result. For example, a member lookup can legitimately return `MEMBER_NOT_FOUND`. This is reported as a business outcome rather than a hard automation failure.

- **Recoverable errors** - Temporary issues, such as network or service failures, are retried using a bounded retry policy. Retry attempts are recorded in the execution evidence.
- **Hard failures** - Errors that prevent safe continuation, such as invalid selectors, missing UI elements, unsupported actions, or persistent failures after retries.

The project also generates evidence files for successful replay and failure cases, including structured JSON logs and screenshots.

UI drift - UI drift is detected when recorded targets can no longer be resolved or the expected checkpoint cannot be verified. The current system fails rather than silently choosing a different element. This prioritizes correctness over attempting an uncertain automatic repair.

4. Heterogeneity & Multi-Tenant

The capability model is separated from the UI automation through Surface abstraction. The current implementation uses a Playwright web surface, while other surfaces could support legacy web or desktop applications using the same capability structure. This separation also enables reuse across institutions, with tenant-specific configuration providing the appropriate endpoint, environment, credentials, and UI configuration.

The capability also separates the workflow from runtime input values. Values such as `member_id` and `initial_deposit` are supplied during replay rather than being hard-coded into the workflow. This allows the same capability structure to be reused with different inputs and environments.

The current prototype supports web applications through Playwright; desktop surfaces, credential isolation, and application-version compatibility are future extensions.

5. Escalation & Handoff

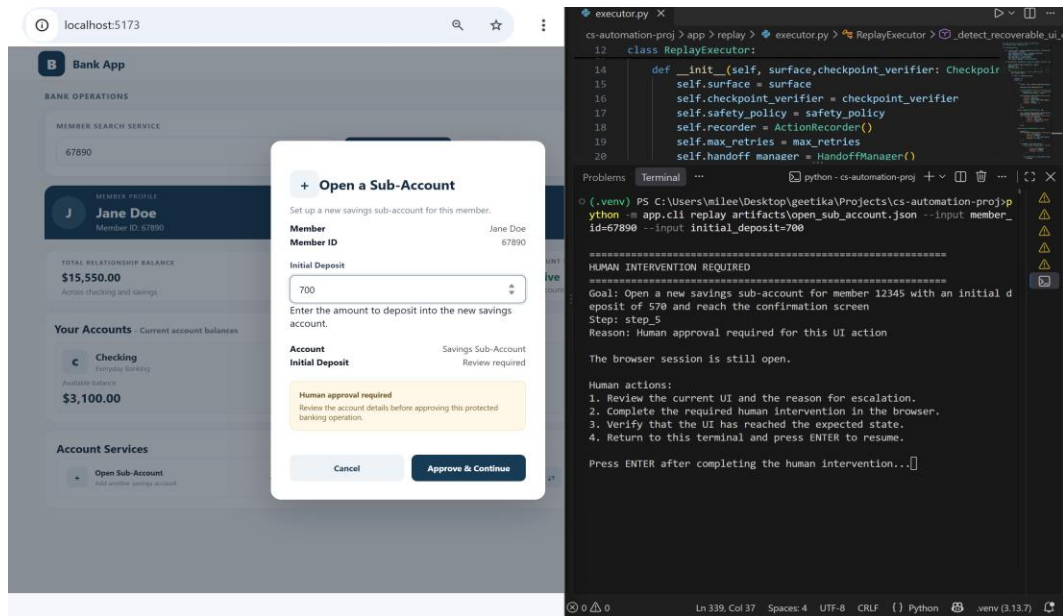
Human intervention is required when the automation encounters an action that should not be performed autonomously.

- The project implements a `SafetyPolicy` and `HandoffManager` for this purpose. When a risky action is encountered, the system creates an intervention request containing info such as intervention ID, original goal, current step, reason for stopping, screenshot of current UI and status.
- The human then takes control of the same live session and performs the required intervention.
- The handoff manager records the human action and changes the request state from pending to approved and then resumed.

For example, the project includes an open-sub-account flow where an approval action is marked: **safety = human_required**. The automation checks the safety attribute and pauses (screenshot-1) instead of performing the approval automatically.

- After the intervention is completed, control is returned to the automation process. The page is observed again before execution continues, so the system does not simply assume that the expected state was reached.

Screenshot -1 Human approval required for a protected banking action during replay.



Screenshot- 2 Replay successfully resumes and completes after human intervention.

```
Press ENTER after completing the human intervention...

=== Replay Result ===
{
  "status": "success",
  "capability_id": "open_sub_account",
  "version": "1.0.0",
  "outputs": {},
  "executed_steps": [
    "step_1",
    "step_2",
    "step_3",
    "step_4",
    "step_5",
    "step_6"
  ]
}
```

The current prototype uses terminal synchronization by **pressing Enter** in terminal (screenshot-2) to signal that the human has completed the intervention. A production implementation could replace this with a dedicated approval UI or session-state mechanism.

6. Safety

Safety is enforced independently from the LLM. The LLM determines what UI action is needed, while SafetyPolicy determines whether that action is allowed. The current model separates actions into:

- **Safe** — ordinary navigation, filling, clicking, extracting, and finishing.
- **Risky** — protected operations that require human approval.
- **Blocked** — operations that automation is not permitted to perform.

In the project the UI elements that require human interaction or blocked contain the **safety metadata** with the **label data-safety**. The observer records this metadata and preserves it when creating capability targets. This allows the application to mark a control for the step. This provides an independent safety layer between the LLM's proposed action and the actual browser operation. The LLM can select an action, but it cannot bypass the safety policy.

The system also avoids storing secrets in capability artifacts. The action recorder includes redaction support for sensitive fields such as passwords, tokens, API keys, authorization information, and secrets.

This is particularly important for banking workflows because an incorrect automated action can have financial consequences. The requires explicit allowlisting, conservative handling of risky actions, and appropriate redaction of sensitive information.

7. Cuts

The implementation deliberately focuses on the core requirement: discover a workflow once, save it as a reusable capability, and replay it deterministically with error handling, safety controls, and human escalation.

With additional development time, the next improvements would be:

1. Add additional surface implementations for legacy web and desktop applications.
2. Add artifact approval and lifecycle management.
3. Complex branching and looping workflows.
4. Build a browser-based operator console for human handoff.
5. Add replay stability testing across many repeated runs.
6. Add bounded LLM-assisted recovery for selected replay failures.